

Numeri casuali

Sequenze che sembrano casuali: il seme, numeri reali e interi in un intervallo, il primo istogramma

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027



SAPIENZA
UNIVERSITÀ DI ROMA

Come si fa «a caso» con un calcolatore?

Il calcolatore esegue formule: i numeri «casuali» sono una successione che sembra casuale

piccolo.c

```c
#include <stdio.h>

int main() {
    int x = 7;          // il seme
    int i;
    for (i = 0; i < 20; i++) {
        printf("%d ", x);
        x = (5 * x + 3) % 16;   // la formula
    }
    printf("\n");
}
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe piccolo.c
$ ./app.exe
7 6 1 8 11 10 5 12 15 14 9 0 3 2 13 4 7 6 1 8
```

Il seme

x = 7: il numero da cui si parte

La formula

(5 x + 3) % 16 dà il numero dopo: è il resto della divisione

Si ripete

dopo 16 numeri si ricomincia: drand48 ha 2^48 stati

drand48(): un numero in [0, 1)

Ogni chiamata dà il numero successivo della successione

casuali.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;
    for (i = 0; i < 10; i++) {
        printf("%.4f\n", drand48());
    }
}
```

[0, 1)

0 compreso, 1 escluso; l'include è <stdlib.h>

Riesegui: gli stessi numeri!

la successione è sempre la stessa

Terminale

```
$ gcc -Wall -Wextra -o app.exe casuali.c
$ ./app.exe
0.3965
0.8405
0.3533
0.4466
0.3187
0.8864
0.0156
0.5841
0.1594
0.3837
$ ./app.exe | head -3
0.3965
0.8405
0.3533
```

Il seme decide la successione

srand48(seme): stesso seme, stessi numeri; seme diverso, altra successione

seme.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    long seme;
    int i;
    printf("seme: ");
    scanf("%ld", &seme);
    srand48(seme);
    for (i = 0; i < 3; i++) {
        printf("%.4f\n", drand48());
    }
}
```

Ripetere un esperimento

stesso seme: stessi risultati, utile per cercare un errore

Terminale

```
$ gcc -Wall -Wextra -o app.exe seme.c
$ ./app.exe
seme: 5
0.5248
0.2729
0.7886
$ ./app.exe
seme: 5
0.5248
0.2729
0.7886
$ ./app.exe
seme: 6
0.3956
0.9774
0.5270
```

`srand48(time(NULL));` l'orologio: da ora lo usiamo sempre

Un numero reale tra a e b

*a + (b - a) * drand48(): si «stira» l'intervallo [0, 1) e lo si sposta*

angolo.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    double a = 10., b = 80.;   // in gradi
    double angolo;
    int i;
    srand48(time(NULL));
    for (i = 0; i < 6; i++) {
        angolo = a + (b - a) * drand48();
        printf("%.2f\n", angolo);
    }
}
```

Terminale: angoli tra 10 e 80

```
$ gcc -Wall -Wextra -o app.exe angolo.c
$ ./app.exe
56.67
19.41
35.24
79.92
59.98
54.18
```

drand48() = 0	drand48() = 0.25	drand48() = 0.5	drand48() = 0.75	drand48() = 1 (escluso)
angolo = 10	**angolo = 27.5**	**angolo = 45**	**angolo = 62.5**	**angolo = 80**

Un numero intero tra n e m: il resto

lrand48() % (m - n + 1) + n: il resto della divisione fa il lavoro

dado.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n = 1, m = 6;     // facce da 1 a 6
    int faccia, i;
    srand48(time(NULL));
    for (i = 0; i < 12; i++) {
        faccia = lrand48() % (m - n + 1) + n;
        printf("%d ", faccia);
    }
    printf("\n");
}
```

Terminale: 12 lanci

```
$ gcc -Wall -Wextra -o app.exe dado.c
$ ./app.exe
4 3 1 5 3 1 4 1 1 2 4 4
```

% 6	+ 1
0 1 2 3 4 5	1 2 3 4 5 6

lrand48()

un intero (long) tra 0 e 2^31 - 1; con % resta tra 0 e m - n

6 facce = m - n + 1

6 resti possibili, da 0 a 5

Una trappola: manca il + 1

Con % (m - n) il valore massimo non esce mai

dado_errore.c (la riga sbagliata in rosso)

```c
int main() {
    int n = 1, m = 6;
    int faccia, i, sei = 0;
    srand48(time(NULL));
    for (i = 0; i < 1000; i++) {
        faccia = lrand48() % (m - n) + n;    // manca + 1
        if (faccia == 6) {
            sei++;
        }
    }
    printf("su 1000 lanci il 6 compare %d volte\n", sei);
}
```

Il programma compila e gira

ma il 6 non esce mai: nessun errore a schermo

Terminale

```
$ gcc -Wall -Wextra -o app.exe dado_errore.c
$ ./app.exe
su 1000 lanci il 6 compare 0 volte
```

% 5	+ 1
0 1 2 3 4	1 2 3 4 5

Controlla gli estremi

conta quanti valori deve avere l'intervallo: m - n + 1, non m - n

Un intero tra n e m, con drand48()

*n + (int)(drand48() * (m - n + 1)): si taglia [0, 1) in m - n + 1 fette uguali*

dado_drand.c

```c
// include come in dado.c
int main() {
    int n = 1, m = 6;    // facce da 1 a 6
    int faccia, i;
    srand48(time(NULL));
    for (i = 0; i < 12; i++) {
        faccia = n + (int)(drand48() * (m - n + 1));
        printf("%d ", faccia);
    }
    printf("\n");
}
```

Terminale: 12 lanci

```
$ gcc -Wall -Wextra -o app.exe dado_drand.c
$ ./app.exe
5 1 3 6 5 4 1 1 4 6 1 4
```

(int) tiene la parte intera

3.9 diventa 3: il cast di prima, ora da double a int



drand48() * 6 è un reale in [0, 6): (int) lo riduce a 0, 1, ..., 5; + n lo sposta a 1, ..., 6

Perché meglio drand48() del resto

Il resto usa le cifre binarie in fondo al numero, che sono le meno casuali

bitbassi.c

```c
// stdio.h come sempre
int main() {
    int x = 7;          // il seme, come in piccolo.c
    int i;
    printf("  x  x%%2  x/8\n");
    for (i = 0; i < 12; i++) {
        // x % 2: ultima cifra; x / 8: prima cifra
        printf("%3d   %d    %d\n", x, x % 2, x / 8);
        x = (5 * x + 3) % 16;
    }
}
```

Il resto: l'ultima cifra

1 0 1 0 1 0 ...: alterna sempre. Il resto della divisione per 2 usa proprio questa cifra

Terminale: il generatore minuscolo

```
$ gcc -Wall -Wextra -o app.exe bitbassi.c
$ ./app.exe
  x  x%2  x/8
  7   1    0
  6   0    0
  1   1    0
  8   0    1
 11   1    1
 10   0    1
  5   1    0
 12   0    1
 15   1    1
 14   0    1
  9   1    1
  0   0    0
```

x / 8: la prima cifra

nessuna regolarità: drand48() usa le cifre in testa

Pari o dispari, testa o croce

drand48() < p dice «sì» con probabilità p: vale per p = 0.5 e per ogni altro p

paridispari.c (dopo gli include)

```c
int main() {
    int i, pari = 0, rivelati = 0;
    double eff = 0.85;          // efficienza del rivelatore
    srand48(time(NULL));
    for (i = 0; i < 20; i++) {
        if (drand48() < 0.5)
            printf("P ");
        else
            printf("D ");
    }
    printf("\n");
    for (i = 0; i < N; i++) {
        if (drand48() < 0.5)     // p = 0.5
            pari++;
        if (drand48() < eff)     // p = eff
            rivelati++;
    }
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe paridispari.c
$ ./app.exe
D D P P D D P D P D D D P D P P D P D D
pari: 49947 su 100000
rivelati: 84990 su 100000
```

Non lrand48() % 2

è la cifra finale: si ripete (slide precedente). Con drand48() < 0.5 si usa la cifra giusta

`drand48() < p` probabilità p; `drand48() >= p` probabilità 1 - p

Intervalli di larghezza diversa

Un solo numero in [0, 7): quanti ne cadono in [0, 1), in [1, 3) e in [3, 7)?



intervalli.c (la parte nuova)

```
double x;
for (i = 0; i < N; i++) {
    x = 7. * drand48();        // un numero in [0, 7)
    if (x < 1)                 // [0, 1): larghezza 1
        n1++;
    else if (x < 3)            // [1, 3): larghezza 2
        n2++;
    else                       // [3, 7): larghezza 4
        n3++;
}
```

Terminale: 100000 numeri in [0, 7)

```
$ gcc -Wall -Wextra -o app.exe intervalli.c
$ ./app.exe
[0, 1): 14453
[1, 3): 28603
[3, 7): 56944
```

Più largo, più punti

i punti sono proporzionali alla larghezza: 1/7, 2/7, 4/7

Contare con if

x < 1, poi x < 3, altrimenti l'ultimo intervallo

Sono davvero «uniformi»?

Si scrivono 100000 numeri su un file: tutti gli intervalli devono avere circa lo stesso numero di punti

uniformi.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 100000

int main() {
    FILE *fp;
    int i;
    srand48(time(NULL));
    fp = fopen("casuali.txt", "w");
    if (fp == NULL) {
        printf("errore: impossibile aprire casuali.txt\n");
        exit(-1);
    }
    for (i = 0; i < N; i++) {
        fprintf(fp, "%.8f\n", drand48());
    }
    fclose(fp);
}
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe uniformi.c
$ ./app.exe
$ wc -l casuali.txt
  100000 casuali.txt
$ head -3 casuali.txt
0.53752549
0.83898944
0.09898509
```

Un numero per riga

solo numeri nel file, perché Python li legga con np.loadtxt

L'istogramma: quanti numeri in ogni intervallo

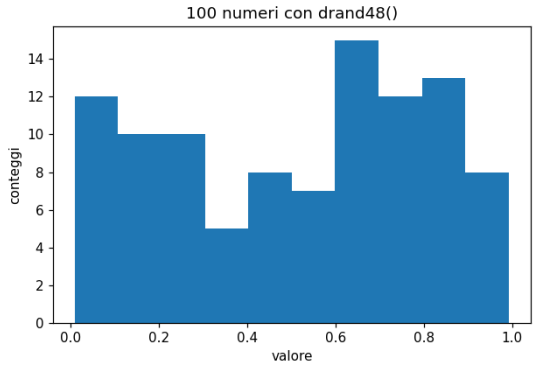
plt.hist(x, bins=10) divide [0, 1) in 10 intervalli uguali e conta

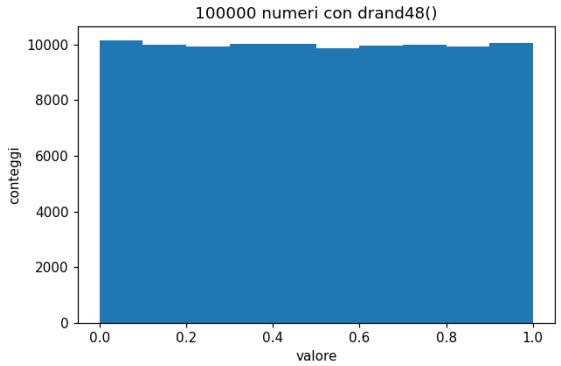
istogramma.py

```
import numpy as np
import matplotlib.pyplot as plt

x = np.loadtxt('casuali.txt')    # uno per riga

plt.hist(x, bins=10)
plt.xlabel('valore')
plt.ylabel('conteggi')
plt.title('100000 numeri con drand48()')
plt.savefig('istogramma.png')
plt.show()
```





plt.hist(x, bins=10)

il primo argomento è l'array dei valori, bins il numero di intervalli

100 numeri

barre molto diverse tra loro: il caso fa grandi fluttuazioni

100000 numeri

barre quasi uguali, tutte tra 9884 e 10157: è uniforme

Le ricette

Da ricordare: seme, reali in [0, 1) e [a, b), interi in [n, m], eventi con probabilità p

cosa si vuole	come si scrive
seme (una volta, all'inizio)	`srand48(seme);`
reale in [0, 1)	`drand48()`
reale in [a, b)	`a + (b - a) * drand48()`
intero in [n, m]	`n + (int)(drand48() * (m - n + 1))`
sì con probabilità p	`drand48() < p`
intero in [n, m], con il resto	`lrand48() % (m - n + 1) + n`

#include <stdlib.h>

per srand48, drand48 e lrand48

time(NULL)

serve #include <time.h>; seme diverso a ogni secondo

Solo Linux e macOS

su Windows queste funzioni non ci sono (sono POSIX)

Esercizi

Da fare a casa, poi si controlla con un programma in C

1

Roulette

Una roulette ha 37 caselle, numerate da 0 a 36. Il programma legge da tastiera il seme, poi simula 15 giri e stampa su una riga i 15 numeri usciti. Alla fine stampa quante volte è uscito lo zero.

per controllare:

```
seme 9: 35 31 5 15 0 ...
uno zero su 15
```

2

Temperature

Simula 8 temperature di una giornata invernale: numeri reali distribuiti in modo uniforme tra -5 e 15 gradi. Stampale su una riga, con un solo decimale e uno spazio tra una e l'altra.

per controllare:

```
ogni valore tra -5.0 e 15.0
per esempio: 8.1 6.4 -4.0 ...
```

3

Tempi

Un cronometro misura tempi distribuiti in modo uniforme tra 2 e 5 secondi. Scrivi 5000 tempi nel file tempi.txt, uno per riga, e stampa il minimo e il massimo estratti. Con Python disegna l'istogramma con 15 intervalli.

per controllare:

```
minimo vicino a 2
massimo vicino a 5
barre quasi uguali
```

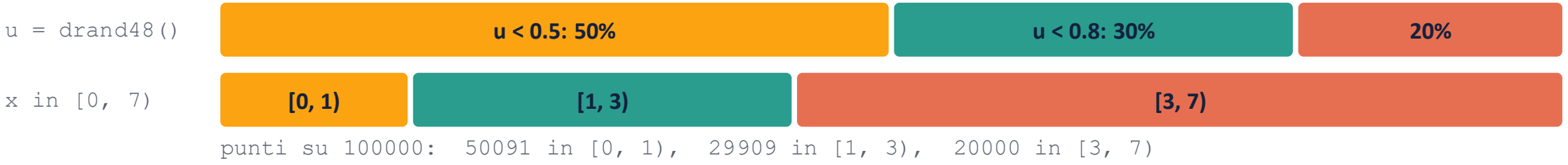
Compila con `gcc -Wall -Wextra -o app.exe nome.c` (python3 tempi.py per il grafico)

Riepilogo

1	**Pseudo-casuali**	una formula e un seme: stesso seme, stessa successione; srand48(seme) una volta sola, time(NULL) per cambiare
2	**Le ricette**	drand48() in [0, 1); a + (b - a) * drand48() in [a, b); n + (int)(drand48() * (m - n + 1)) tra n e m; drand48() < p con probabilità p
3	**Verificare**	scrivere i numeri su file e disegnare l'istogramma con plt.hist: più numeri, più piatto

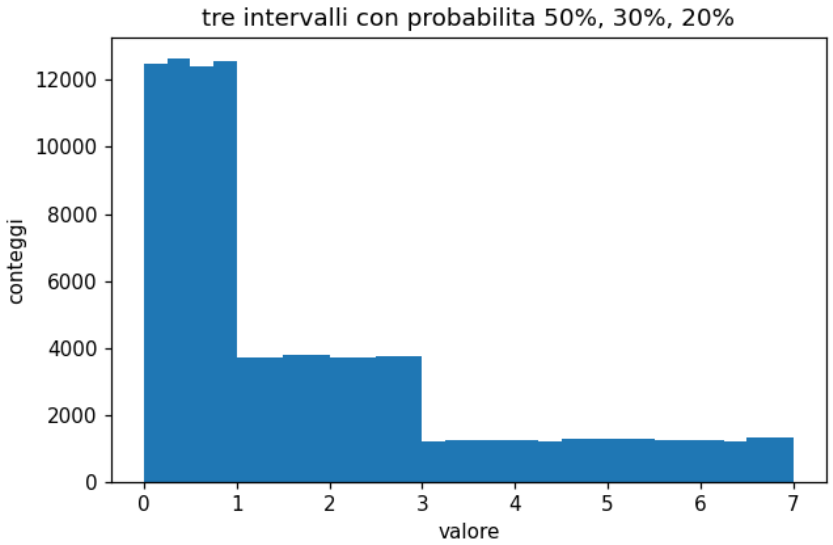
Per i più curiosi: probabilità a scelta

Una estrazione sceglie l'intervallo, un'altra il punto dentro



intervalli_prob.c (la scelta)

```
u = drand48();                  // intervallo
if (u < 0.5)                    // 50%: [0, 1)
    x = 0. + 1. * drand48();
else if (u < 0.8)               // 30%: [1, 3)
    x = 1. + 2. * drand48();
else                            // 20%: [3, 7)
    x = 3. + 4. * drand48();
fprintf(fp, "%.8f\n", x);
```



`0.5, 0.8` le soglie sono probabilità cumulate: 0.5, poi 0.5 + 0.3